LECTURE 1

Introduction to Mobile Applications Development

Mobile computing · Cross-platform development · React Native · JavaScript & JSX

React Native Track

University of Mindanao Tagum College

01

Mobile Computing

What changes when the computer fits in a pocket

DEFINITION

Three pillars hold it up

A computing model where the device, the user, and the network connection can all move — and the system is expected to keep working anyway.

HW

Mobile hardware

Phones, tablets, wearables, embedded and IoT devices — the physical thing in the hand.

SW

Mobile software

The operating system and the applications built to run on that hardware.

NET

Mobile communication

The connectivity layer: cellular 4G and 5G, Wi-Fi, Bluetooth, NFC.

WHAT MAKES MOBILE DIFFERENT

Every constraint is a design consequence

- **Small screen, touch input**
  No hover state. Tap targets 44–48dp minimum.
- **Battery is finite**
  Background work and GPS polling get you killed by the OS.
- **Network is unreliable**
  Assume offline. Cache. Never assume a request completes.
- **Limited RAM**
  The OS can terminate you — save and restore state.
- **Sensors and hardware**
  Camera, GPS, accelerometer, biometrics, NFC.
- **Permissions and privacy**
  Users consent, and can revoke. Handle refusal gracefully.

THE LANDSCAPE

Two platforms. Two of everything.

Android

Google · AOSP

- Linux-based, open source
- Dominant global market share
- Huge device fragmentation
- Kotlin (Java legacy)
- Play Store + sideloading

iOS

Apple · closed

- Tightly controlled hardware range
- Strong revenue per user
- Few devices, easy targeting
- Swift (Objective-C legacy)
- App Store only

Two platforms, two languages, two toolchains, two teams. That problem is the entire reason cross-platform development exists.

CATEGORIES

Four ways to put an app on a phone

A	**Native**	Built with the platform's own SDK and language. Kotlin for Android, Swift for iOS.
B	**Cross-platform**	One codebase, real native UI on both platforms. React Native, Flutter.
C	**Hybrid**	A website wrapped in an app shell. Cordova, Ionic, Capacitor.
D	**Progressive Web App**	A website with offline support and an install prompt. No app store needed.

CATEGORIES

Four ways to put an app on a phone

A	**Native**	Built with the platform's own SDK and language. Kotlin for Android, Swift for iOS.
B	**Cross-platform**	One codebase, real native UI on both platforms. React Native, Flutter.
C	**Hybrid**	A website wrapped in an app shell. Cordova, Ionic, Capacitor.
D	**Progressive Web App**	A website with offline support and an install prompt. No app store needed.

Option B is where we live for the rest of this course.

02

Cross-Platform Development

One codebase, two app stores — and what it costs you

THE PROBLEM STATEMENT

Build the same app for Android and iOS

The native route

- 2 codebases
- 2 languages
- 2 skill sets
- 2× the bugs
- Features drift out of sync

The cross-platform route

- 1 codebase
- 1 language
- 1 team
- 1 set of bugs, 1 fix
- Both platforms ship together

BENEFITS

What you gain

70–95%

Code reuse

Typical share of code that runs on both platforms untouched.

1 team

Lower cost

One skill set instead of two. Smaller team, smaller payroll.

Same day

Faster to market

Features land on Android and iOS simultaneously.

1 fix

Easier maintenance

One implementation means one bug, and one place to fix it.

TRADE-OFFS

What it costs you

Performance ceiling	Fine for most apps. A real bottleneck for heavy graphics, AR, or real-time video.
Delayed platform features	A brand-new iOS API may need a native module, or a wait for framework support.
Abstraction leaks	Sooner or later you WILL write platform-specific code. Cross-platform is not platform-ignorant.
Larger app size	You ship a runtime or an engine alongside your own code.
Dependency on maintainers	If the framework stalls, you inherit the problem.

UNDER THE HOOD

The difference is what draws the pixels

Hybrid

Your HTML/CSS/JS

▼

WebView

▼

Operating system

Cheapest. Looks and feels like a website — scrolling gives it away.

React Native

Your JS / JSX

▼

Hermes + JSI

▼

REAL native widgets

A <Text> becomes a UILabel on iOS and a TextView on Android.

Flutter

Your Dart code

▼

Compiled ARM

▼

Skia paints every pixel

Does not use native widgets at all. Pixel-identical everywhere.

React Native BORROWS the platform's components. Flutter REDRAWS them. Hybrid AVOIDS them.

FRAMEWORK COMPARISON

Side by side

	React Native	Flutter	Ionic	Native
Language	**JavaScript / TS**	Dart	HTML/CSS/JS	Kotlin / Swift
Backed by	**Meta**	Google	Ionic	Google / Apple
UI rendering	**Native components**	Own engine	WebView	Native
Performance	**High**	High	Moderate	Highest
Learning curve	**Low if you know JS**	Moderate	Lowest	Steepest
Hiring pool	**Very large**	Smaller	Large	Specialised

Why React Native for this course: you learn one new paradigm, not one new language.

GROUP ACTIVITY · 8 MINUTES

Pick a stack. Defend it in 30 seconds.

A A mobile 3D racing game

B An internal HR leave-request app, 200 staff

C A banking app with biometrics and strict security review

D A food delivery MVP — 3 developers, 8-week deadline

Each group gets one scenario. Choose native, React Native, Flutter, hybrid or PWA — then say why in one sentence.

03

React Native

Learn once, write anywhere — note carefully what that does NOT say

WHAT IT IS

An open-source framework from Meta

React Native lets you build genuinely native mobile apps using React and JavaScript. Released in 2015, it brought React's declarative component model to iOS and Android.

2015

Released by Meta.
Still actively developed.

"Learn once, write anywhere."

Note carefully — NOT "write once, run anywhere". Your skills transfer. Not every line does.

Shipping React Native today: Instagram · Discord · Shopify · Coinbase · Microsoft Office mobile · Bloomberg

HOW IT WORKS

Three pieces, talking to each other

1	**Your JavaScript**	Components, business logic, state — everything you actually write.
2	**The JS engine (Hermes)**	Meta's engine, optimised for mobile startup time and memory use.
3	**The native layer**	Real UIView and android.view objects rendered by the platform.

The old Bridge was two people passing written notes under a door. The New Architecture — JSI, Fabric, TurboModules — knocked the door down. Now they just talk.

CORE COMPONENTS

You get primitives, not HTML tags

React Native	Web equivalent	Becomes natively
`<View>`		UIView / ViewGroup
`<Text>`	<p>,	UILabel / TextView
`<Image>`	<img>	UIImageView / ImageView
`<TextInput>`	<input>	UITextField / EditText
`<ScrollView>`	scrolling div	UIScrollView / ScrollView
`<FlatList>`	— none —	Recycling virtualised list
`<Pressable>`	<button>	Touch-handling native view

Beginner error #1: all text must sit inside <Text>. You cannot put a bare string in a <View>.

ECOSYSTEM

The tools around the framework

Expo	A managed toolchain and SDK. Handles builds and native modules — and lets you run the app on your own phone by scanning a QR code. No Xcode or Android Studio needed to start.
React Navigation	Screen navigation: stacks, tabs and drawers. You will use this in the assignment.
npm + Metro	The same package registry you use on the web, and the JavaScript bundler that packs your app.
Fast Refresh	Save the file, see the change in under a second — without losing your app state.

For this course we use Expo. It removes almost all of the setup pain.

LIVE DEMO

From nothing to running on your phone

```
npx create-expo-app@latest DemoApp
cd DemoApp
npx expo start
```

Fallback if the Wi-Fi dies:
snack.expo.dev — runs entirely in the browser, nothing to install.

1. Scan the QR code with Expo Go — it runs on real hardware
2. Open app/index.tsx, change a string, hit save
3. Watch Fast Refresh update instantly
4. Change a colour in the styles. Save again.
5. Show the same code on Android and iOS side by side

04

JavaScript & JSX

The language is not the hard part. The mental model is.

REFRESHER

The JavaScript you actually need

```
// const by default, let when reassigning. Never var.
const name = "Juan";
let count = 0;

// Arrow functions + template literals
const double = (n) => n * 2;
const greet = (n) => `Hello, ${n}!`;

// Destructuring and spread
const student = { name: "Ana", year: 3 };
const { name: who, year } = student;
const updated = { ...student, year: 4 };

// Async
async function load() {
  const res = await fetch(url);
  return await res.json();
}
```

The three array methods

`map()`

transform every item

`nums.map(n => n * 2)`

`filter()`

select some items

`nums.filter(n => n > 2)`

`reduce()`

collapse to one value

`nums.reduce((a,b) => a+b)`

map() is how you render every list.

JSX

JavaScript that looks like markup

JSX is not HTML and not a template language. It compiles to plain function calls.

```
const element = <Text>Hello</Text>;
// compiles to roughly:  React.createElement(Text, null, "Hello")
```

One root

Return exactly one root element — or wrap in a fragment <>…</>

Curly braces

Embed JavaScript with { } → <Text>{user.name}</Text>

camelCase

Attributes are camelCase: onPress, numberOfLines — not onclick

Close everything

Every tag must close. Self-closing is fine: <Image />

COMPONENTS, PROPS, STATE

A component is a function that returns UI

```
import { useState } from 'react';
import { View, Text, Pressable } from 'react-native';

// Props are inputs — read-only, from the parent
function Greeting({ name }) {
  return <Text style={styles.title}>Hello, {name}!</Text>;
}

// State changes over time and triggers a re-render
export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <View style={styles.container}>
      <Greeting name="Class" />
      <Text>You tapped {count} times</Text>
      <Pressable onPress={() => setCount(count + 1)}>
        <Text>Tap me</Text>
      </Pressable>
    </View>
  );
}
```

Props flow DOWN

A child can never change its parent's props.

State triggers re-render

You never update the UI by hand. You update state; React redraws.

Never mutate state

count++ does nothing. setCount(count + 1) is the only way.

STYLING & LAYOUT

Styles are JavaScript objects, not CSS files

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 24,
  },
  title: { fontSize: 24, fontWeight: 'bold' },
});
```

Four differences from CSS

- camelCase — backgroundColor, not background-color
- No units — numbers are density-independent pixels
- Percentages are strings: width: '50%'
- Flexbox is the layout system, and it is on by default

Two flexbox differences from the web:

`flexDirection` defaults to `'column'`, not `'row'` · `flex` takes a single number, e.g. `flex: 1`

FLATLIST

Rendering a list the right way

```
const courses = [
  { id: '1', code: 'IT 311', title: 'Mobile App Dev' },
  { id: '2', code: 'IT 312', title: 'Systems Integration' },
];

<FlatList
  data={courses}
  keyExtractor={(item) => item.id}
  renderItem={({ item }) => (
    <View style={{ padding: 16 }}>
      <Text style={{ fontWeight: 'bold' }}>{item.code}</Text>
      <Text>{item.title}</Text>
    </View>
  )}
/>
```

Why not map() inside a ScrollView?

FlatList renders only what is on screen and recycles rows as you scroll.

10,000 items

ScrollView will freeze the app.
FlatList will not.

keyExtractor is not optional.

HANDS-ON · 25 MINUTES

Build an "About Me" screen

1. Display your name and course in styled <Text> components
2. Add an <Image> — any avatar or logo
3. List at least three hobbies using <FlatList>
4. Add a <Pressable> that toggles a "fun fact" with useState
5. Centre everything using flexbox

You need

Node LTS · VS Code · Expo Go on your phone

Or just open snack.expo.dev — nothing to install.

Errors to expect

- Bare text not wrapped in <Text>
- Missing keyExtractor on FlatList
- Using count++ instead of the setter
- CSS-style names like background-color

RECAP

Cold call — do not just ask "any questions?"

- **Q1** Name two constraints that make mobile different from desktop.
- **Q2** What does React Native render — native widgets, or its own drawings?
- **Q3** How does Flutter differ in that respect?
- **Q4** What is the difference between props and state?
- **Q5** Why FlatList instead of ScrollView for a long list?

ASSIGNMENT

Turn it into a two-screen app

Extend your "About Me" screen using React Navigation.

Screen 1

Your profile — name, course, image, hobby list

Screen 2

A detail view for one hobby, opened by tapping a list item and receiving the hobby as a navigation param

Submit: a GitHub repository link + a screen recording or two screenshots

Next session: full development environment setup, project structure, and React Navigation in depth.